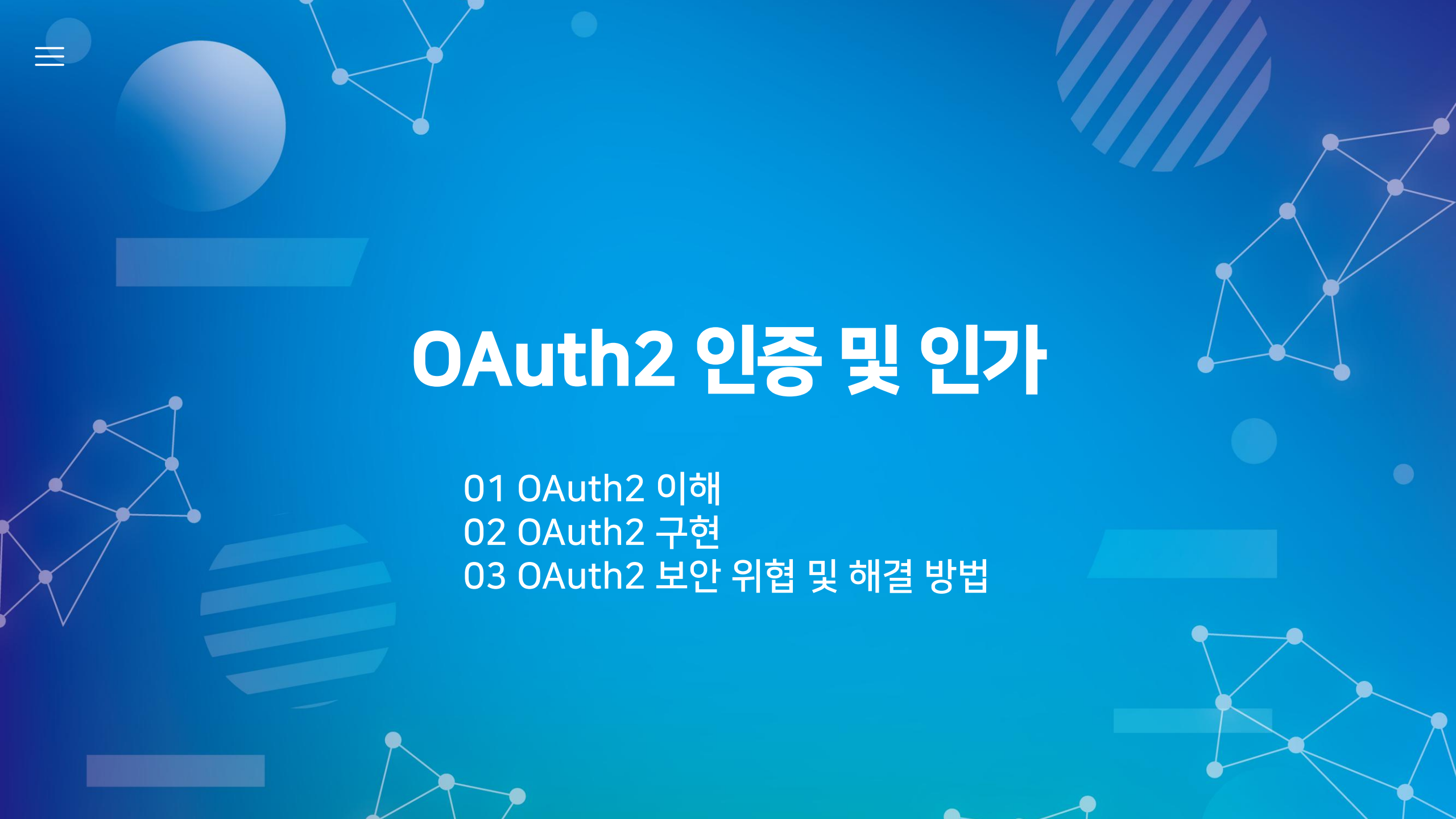


The background is a vibrant blue gradient. It features several abstract elements: a large solid blue sphere in the upper left, a smaller sphere with diagonal stripes in the upper right, and another striped sphere in the lower left. Scattered throughout are network diagrams consisting of white dots connected by thin white lines, forming various geometric shapes like triangles and polygons. There are also solid blue rectangular bars of different sizes and orientations. In the top left corner, there is a small white icon consisting of three horizontal lines.

OAuth2 인증 및 인가

- 01 OAuth2 이해
- 02 OAuth2 구현
- 03 OAuth2 보안 위협 및 해결 방법



학습 목표

- ✓ OAuth2 개념과 필요성을 이해하고 설명할 수 있다.
- ✓ OAuth2 프로세스 및 구성요소를 이해하고, 구현할 수 있다.
- ✓ OAuth2 보안 위협을 이해하고 이에 대한 해결 방안을 설명할 수 있다.



학습 내용

1. OAuth2 개념

2. OAuth2 구현

3. OAuth2 보안 위협 및 해결방법



OAuth2 인증 및 인가

01 OAuth2 이해

02 OAuth2 구현

03 OAuth2 보안 위협 및 해결 방법

OAuth2 란?

정의

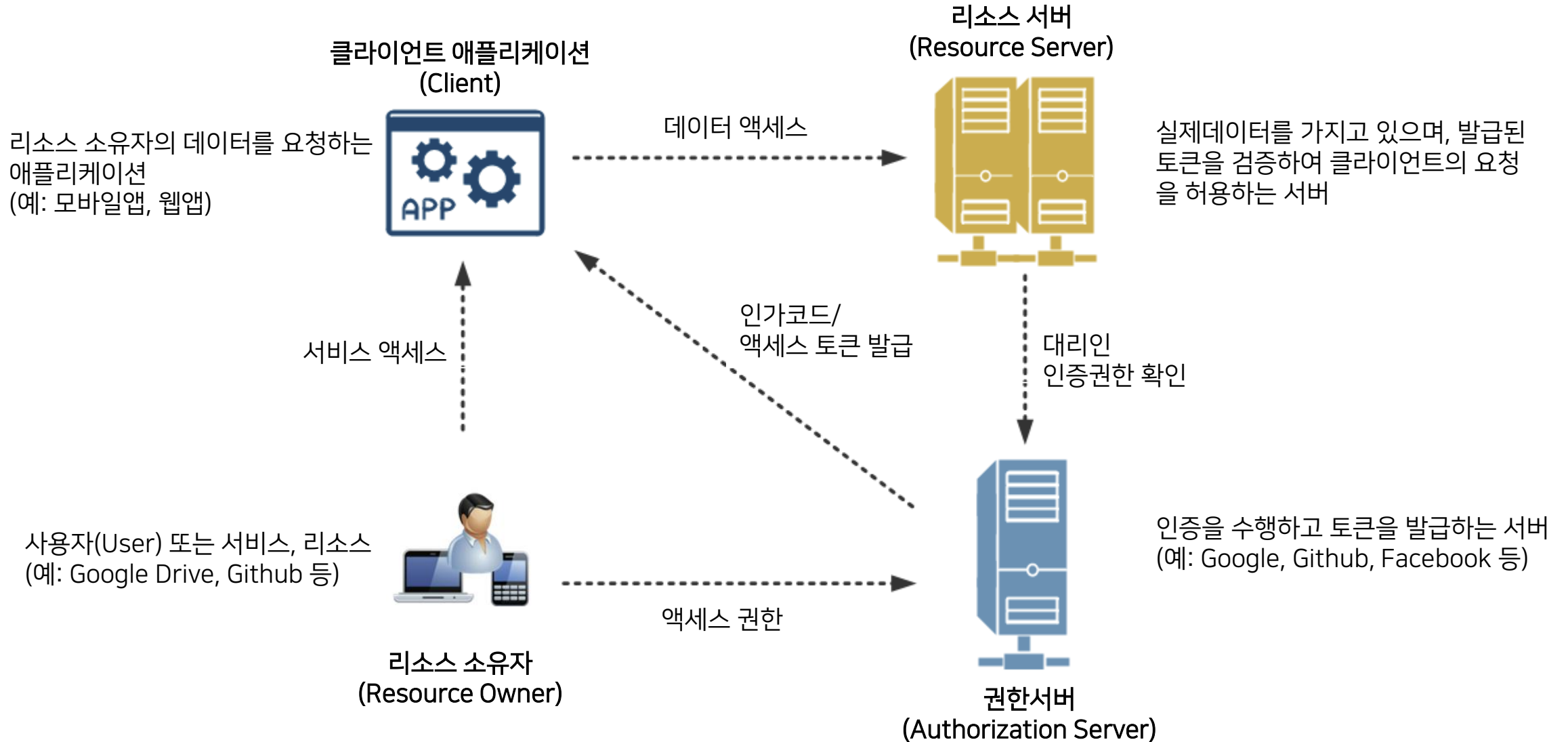
OAuth(Open Authorization)는 클라이언트 애플리케이션이 특정 리소스 소유자를 대신하여 서버 리소스에 액세스할 수 있는 권한 부여를 위한 개방형 표준.

사례)

사용자가 Google 계정을 이용해서 타사앱(B사의 애플리케이션)에 로그인하거나 데이터 공유
사용자가 GitHub 계정을 이용하여 CI/CD 툴(Jenkins)과 연동



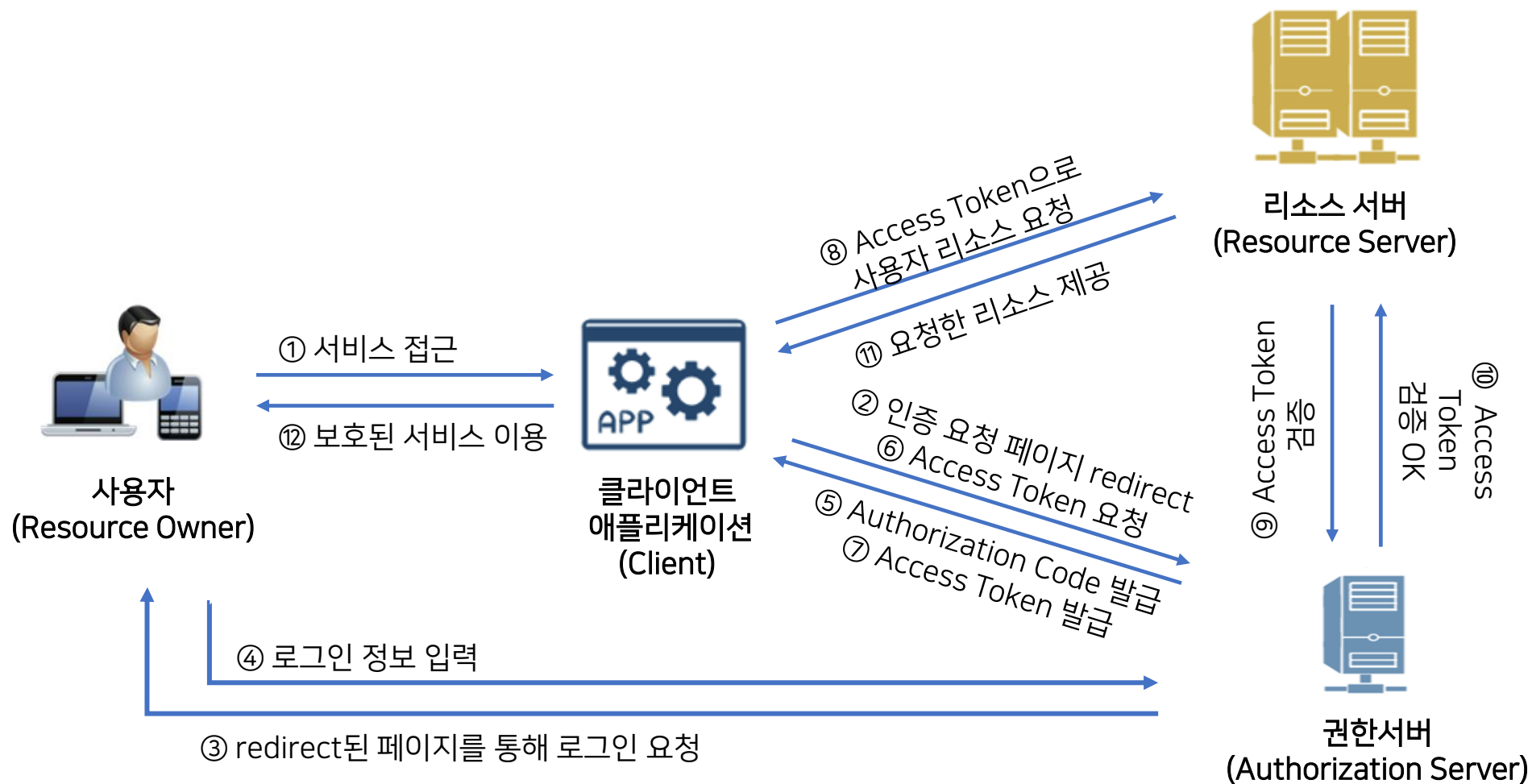
OAuth2 주요 구성요소



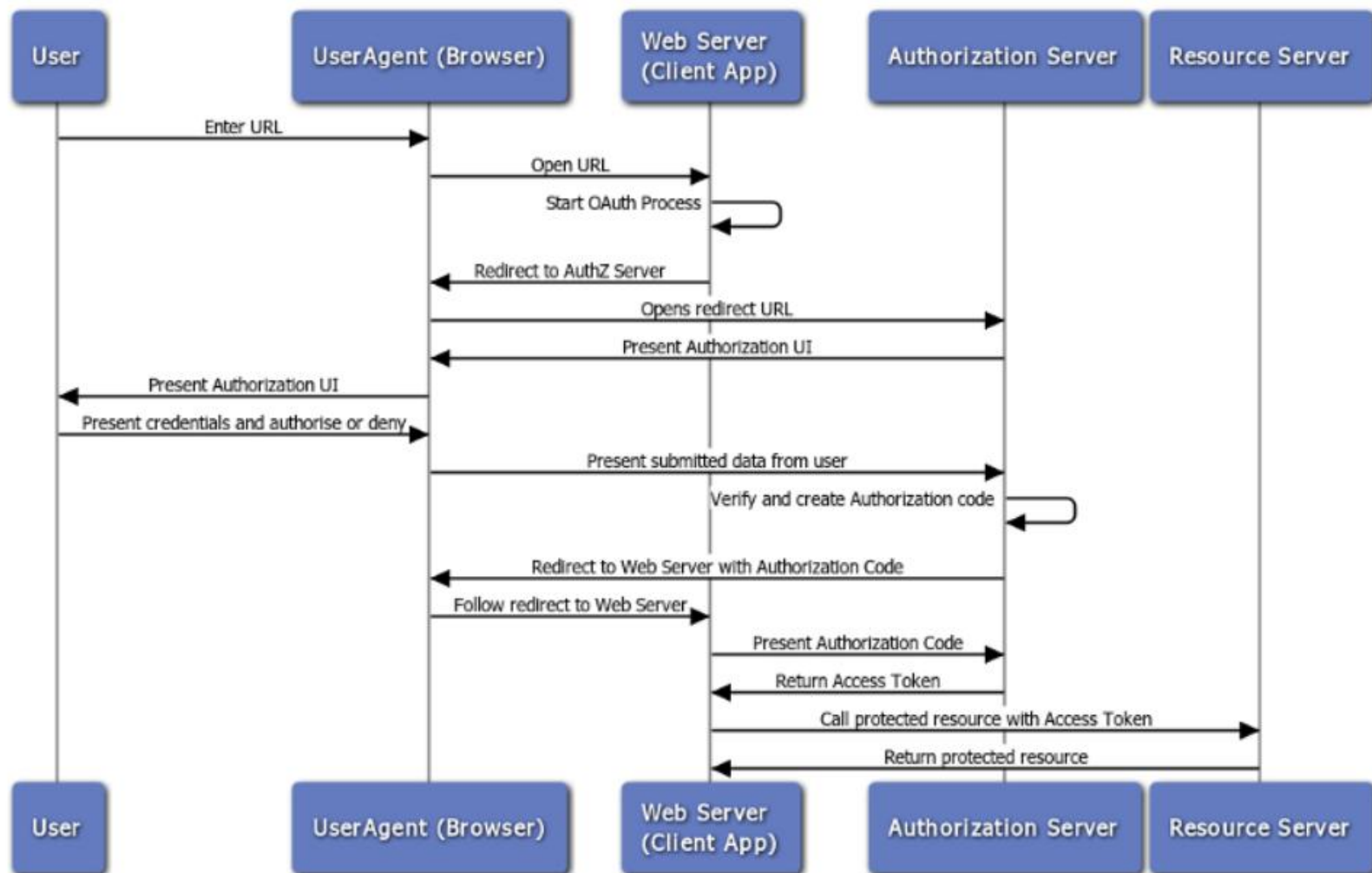
OAuth2 구성 요소

참여자	설명	역할
클라이언트 애플리케이션 (Client Application)	리소스 소유자를 대신해 보호된 리소스에 접근하려는 애플리케이션	사용자 인증 및 권한 부여를 받아, Access Token을 획득하여 리소스 서버에 접근
리소스 소유자 (Resource Owner)	리소스(데이터)를 소유한 사용자	자신의 리소스에 접근할 수 있도록 권한을 부여
권한 서버 (Authorization Server)	사용자 인증과 권한 부여를 처리하고, Access Token을 발급하는 서버	클라이언트의 인증 요청을 검증하고, 적절한 Authorization Grant 또는 Access Token을 발급
리소스 서버 (Resource Server)	보호된 리소스를 보유하고 있는 서버	클라이언트가 제출한 Access Token의 유효성을 확인한 후, 접근 요청에 따라 리소스를 제공

OAuth2 인증 흐름 : Authorization Code Grant



OAuth2 인증 흐름 : Authorization Code Grant



OAuth2 의 장단점

OAuth2의 장점

- 비밀번호 노출 위험 감소

사용자 비밀번호를 클라이언트에 전달하지 않고, 권한 서버에서만 관리하므로 보안성이 향상됨

- SSO (Single Sign-On) 지원

한 번의 로그인으로 여러 애플리케이션에 접근할 수 있어 사용자 편의성이 높아짐

- 토큰 기반 인증

Access 토큰과 Refresh 토큰을 활용하여, 세션 관리 및 권한 검증을 효과적으로 수행,
유효기간을 짧게 운영하여 토큰 탈취 피해가 발생하더라도 피해를 최소화함

- 다양한 클라이언트 지원

웹, 모바일, API 등 여러 클라이언트에 대해 유연하게 적용 가능하며, 확장성이 뛰어남

- 외부 인증 연동 용이

소셜 로그인 및 다른 제3자 인증 시스템과 쉽게 통합

OAuth2 의 장단점

OAuth2 단점

- 구현 복잡성

다양한 인증 흐름(Authorization Code, Implicit, Client Credentials 등)과 토큰 관리가 복잡하여 잘못 구현될 경우 보안 취약점이 발생할 수 있음

- 클라이언트 비밀 관리 문제

모바일이나 공개 클라이언트의 경우, 태생적으로 클라이언트에 보관하는 토큰 등을 완벽하게 보호하기 어려움

- 표준 해석의 차이

OAuth2는 표준 자체가 다소 유연하게 되어 있어서, 연동 시스템 간 구현의 차이 발생할 수 있음

- 추가 보안 고려 필요

PKCE, 토큰 취소(Revocation), 타임스탬프/Nonce 등 추가 보안 대책이 없으면 리플레이 공격, 토큰 탈취 등의 위험이 존재

- 서버 간 신뢰 관계 구성 필요

인증 서버와 리소스 서버 간의 신뢰가 깨지지 않도록 보안 구성이 탄탄하지 않으면 전체 시스템 보안이 위협할 수 있음

OAuth 와 OAuth2 비교

구분	OAuth	OAuth2
보안 방식	· 서명 기반 인증: 각 요청마다 HMAC-SHA1 등으로 서명을 생성하여 전송 · 별도의 전송 보안 없이도 일정 수준의 보안 제공	· 토큰 기반 인증: Access Token과 Refresh Token을 활용 · HTTPS/TLS를 기본 전제 보안 수단으로 사용하여 전송 보안을 확보
구현 복잡성	· 서명 계산 및 검증 과정으로 인해 구현 난이도가 높음 · 클라이언트와 서버 모두에서 복잡한 암호화 로직 필요	· 서명 계산 없이 토큰을 사용하기 때문에 구현이 간단함 · 다양한 인증 흐름(Authorization Code, Implicit, Client Credentials, Resource Owner Password Credentials 등) 지원
유연성 및 확장성	· 표준이 다소 엄격하여 확장에 한계가 있음 · 일부 레거시 시스템에서 주로 사용	· 현대 애플리케이션 환경(웹, 모바일, API 등)에 최적화된 유연한 설계 · 추가 보안 강화 (예: PKCE)를 통해 공개 클라이언트 보안 강화 가능



OAuth2 인증 및 인가

01 OAuth2 이해

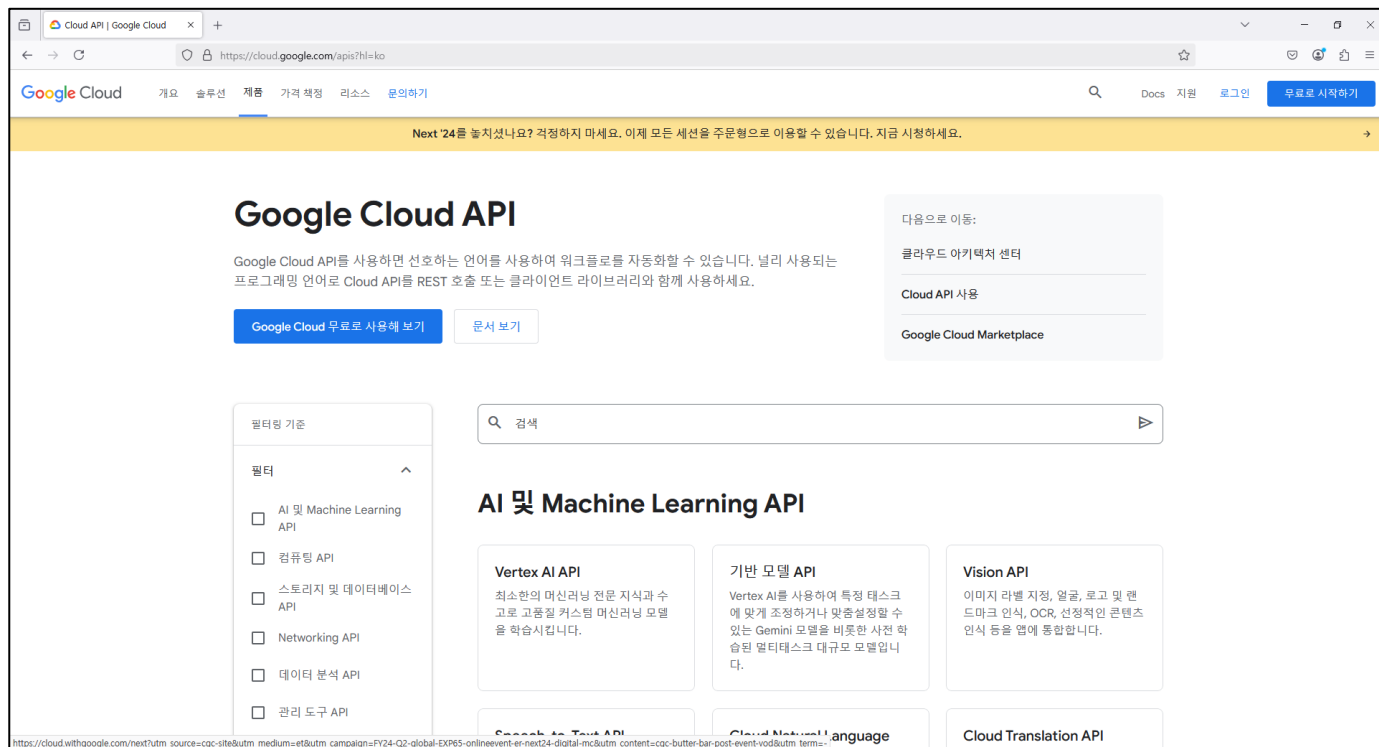
02 OAuth2 구현

03 OAuth2 보안 위협 및 해결 방법

Google OAuth 클라이언트 생성

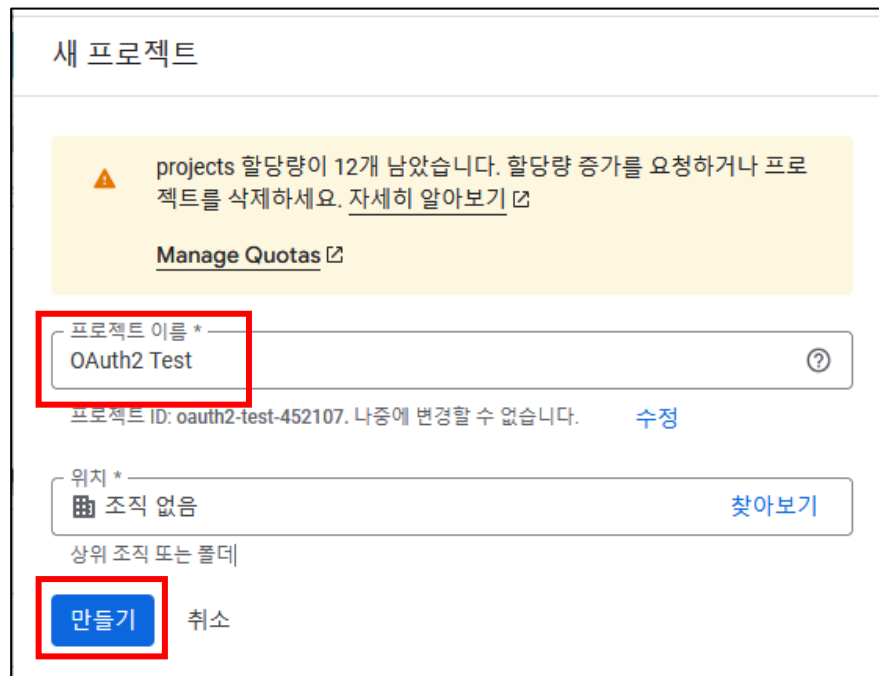
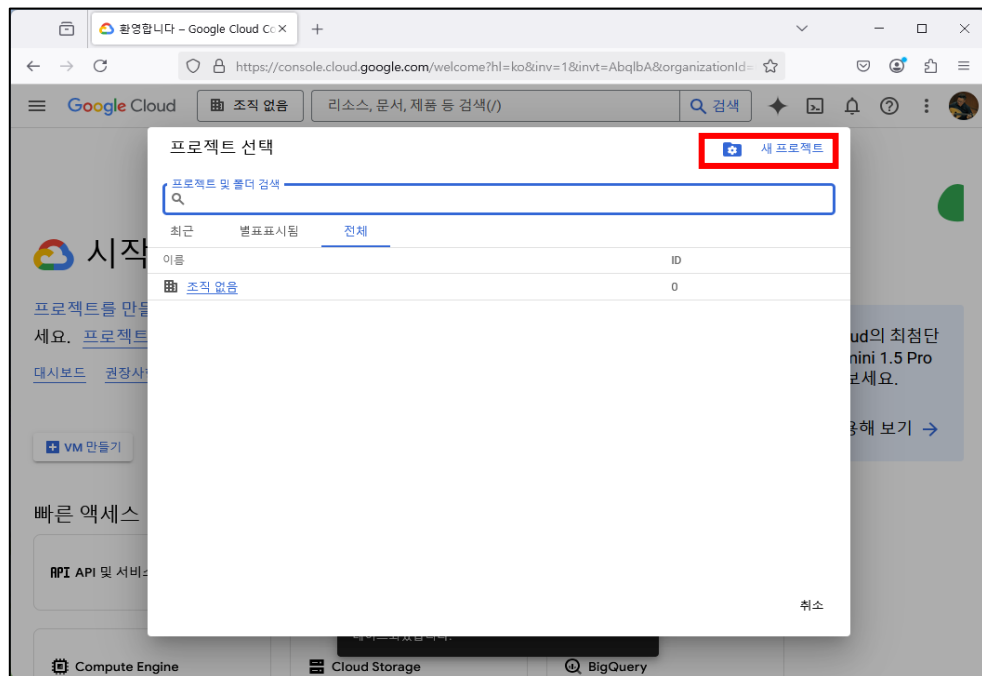
1. Google Cloud 가입 필요

<https://cloud.google.com/apis?hl=ko>



Google OAuth 클라이언트 생성

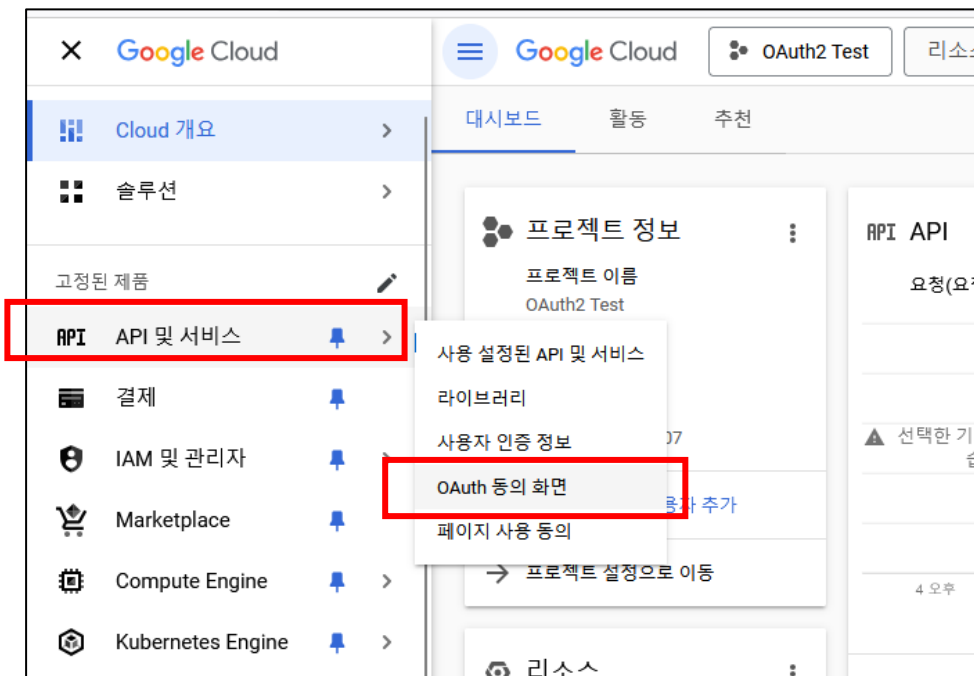
2. 새 프로젝트 생성



Google OAuth 클라이언트 생성

3. 서비스 동의 및 시작

API 및 서비스 > OAuth 동의 화면 선택



Google 인증 플랫폼에서 아직 구성하지 않았을 경우 '시작하기' 눌러 설정



Google OAuth 클라이언트 생성

4. 프로젝트 구성

앱 정보 > 앱 이름 : OAuth2 Test, 사용자 지원 이메일: 개인 메일 주소

프로젝트 구성

1 앱 정보

앱 이름 *
OAuth2 Test
동의를 요청하는 앱의 이름

사용자 지원 이메일 *
pineful@gmail.com
사용자가 동의에 대해 문의하는 곳도입니다. [자세히 알아보기](#)

다음

2 대상 |

3 연락처 정보

4 완료

만들기 취소

대상 > 외부 선택

Google Cloud OAuth2 Test 동의

Google 인증 플랫폼 / 개요 / 브랜딩 만들기

개요

브랜딩

대상

클라이언트

데이터 액세스

인증 센터

프로젝트 구성

1 앱 정보

2 대상

☒ 외부 ?

조직 내 사용자만 사용할 수 있습니다. 인증을 위해 앱을 제출할 필요는 없습니다. [사용자 유형 자세히 알아보기](#)

Google 계정이 있는 모든 테스트 사용자가 사용할 수 있습니다. 앱이 테스트 모드로 시작되고 테스트 사용자 목록에 추가된 사용자에게만 제공됩니다. 앱을 프로젝트에 푸시할 준비가 되면 앱을 인증해야 할 수도 있습니다. [사용자 유형 자세히 알아보기](#)

다음

3 연락처 정보

4 완료

Google OAuth 클라이언트 생성

4. 프로젝트 구성

연락처 정보: 개인 메일 주소

The screenshot shows the '프로젝트 구성' (Project Setup) page in the Google Cloud console. The left sidebar contains a navigation menu with '개요' (Overview) selected. The main content area shows a progress list with four steps: '앱 정보' (App Info), '대상' (Audience), '연락처 정보' (Contact Information), and '완료' (Complete). The '연락처 정보' step is currently active and highlighted with a red box. Below this step, there is a text input field labeled '이메일 주소 *' (Email address *) with the value 'pineful@gmail.com' entered. A red box highlights the input field and the '다음' (Next) button below it. At the bottom of the page, there are '만들기' (Create) and '취소' (Cancel) buttons.

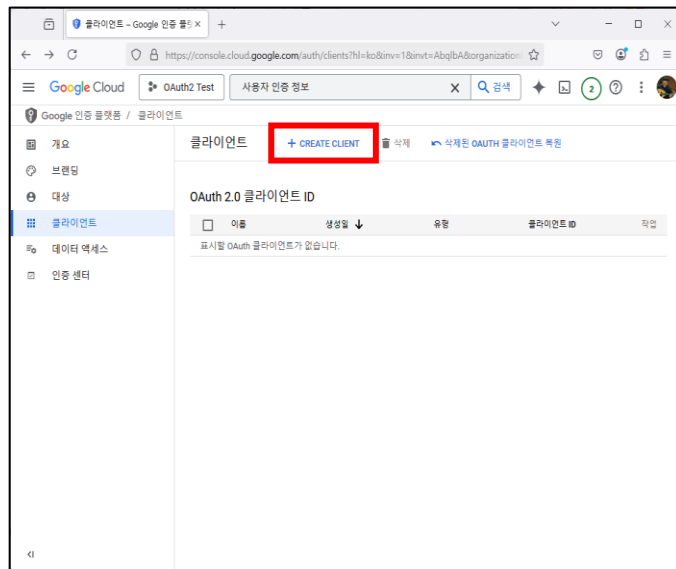
'만들기' 버튼 선택

The screenshot shows the same '프로젝트 구성' (Project Setup) page in the Google Cloud console. The progress list on the left now shows '완료' (Complete) as the final step, and the '연락처 정보' step is no longer active. A red box highlights the '만들기' (Create) button at the bottom of the page, which is located next to the '취소' (Cancel) button. The '다음' (Next) button from the previous step is no longer visible.

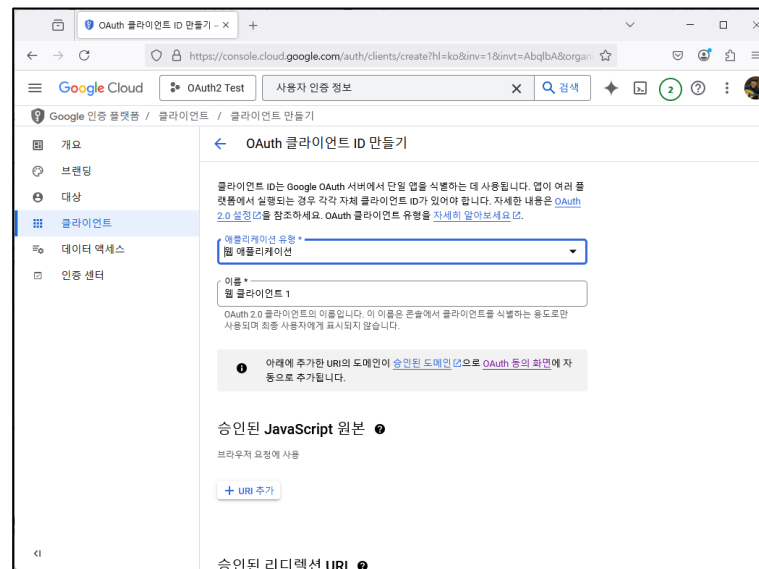
Google OAuth 클라이언트 생성

5. 클라이언트 만들기

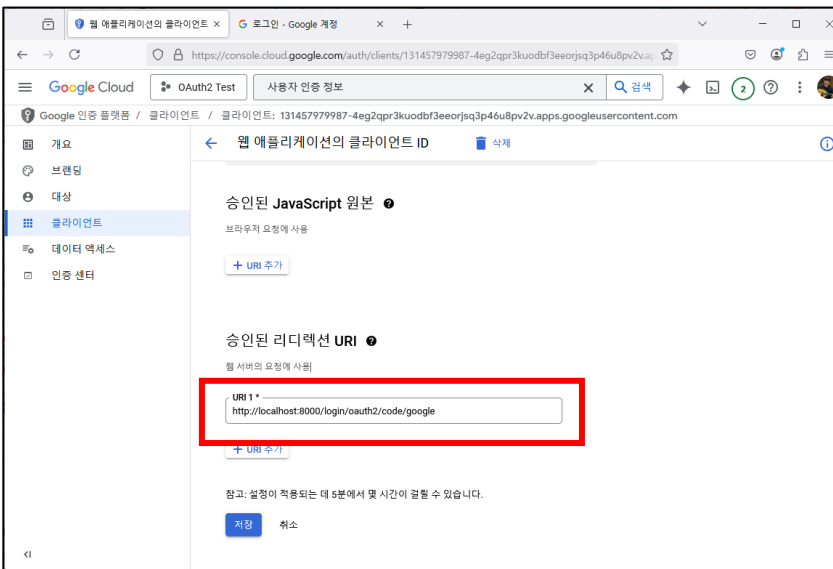
클라이언트 > + CREATE CLIENT 선택



OAuth 클라이언트 ID 만들기



승인된 리디렉션 URI' 항목에 아래 URI 입력



애플리케이션 유형을 '웹 애플리케이션'으로 선택,
이름 디폴트 그대로

http://localhost:8000/login/oauth2/code/google

Google OAuth 클라이언트 생성

5. 클라이언트 만들기(계속)

OAuth 2.0 클라이언트 ID 생성 확인

The screenshot shows the Google Cloud console interface. The left sidebar contains navigation links: 개요, 브랜딩, 대상, 클라이언트 (highlighted), 데이터 액세스, and 인증 센터. The main content area is titled '클라이언트' and includes a '+ CREATE CLIENT' button and a '삭제' button. Below this, there is a section for 'OAuth 2.0 클라이언트 ID' with a table listing the created client.

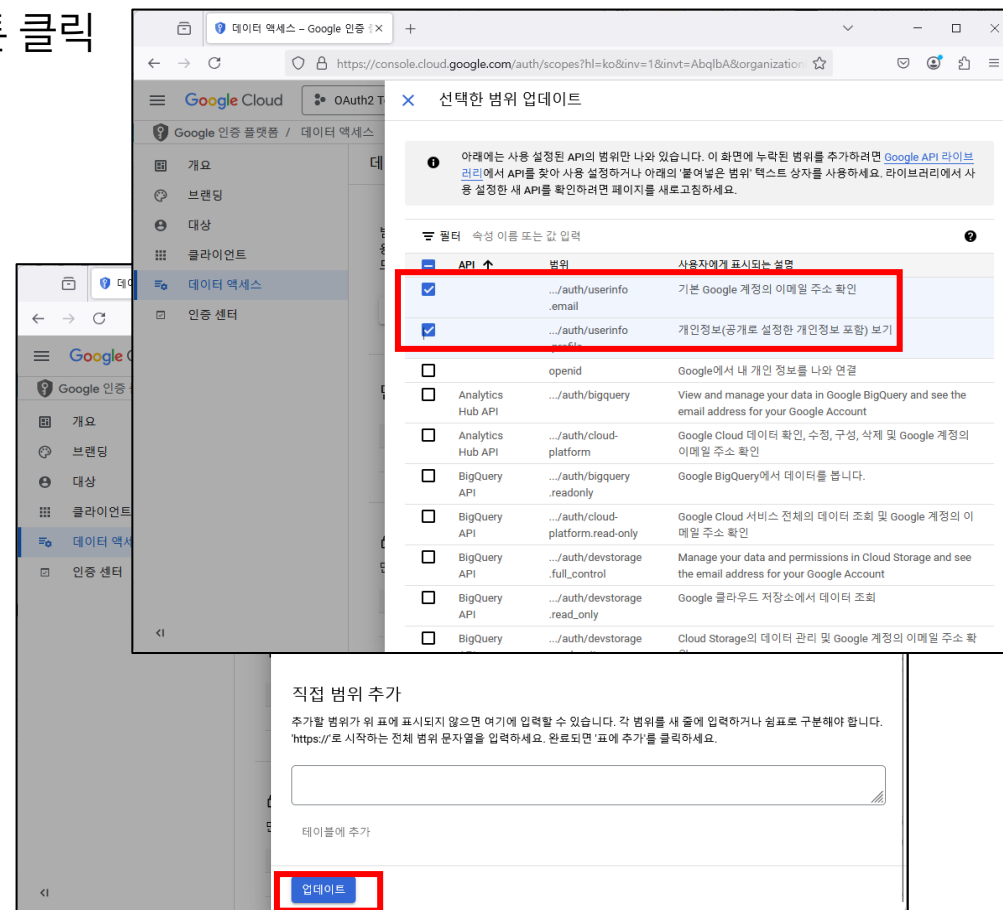
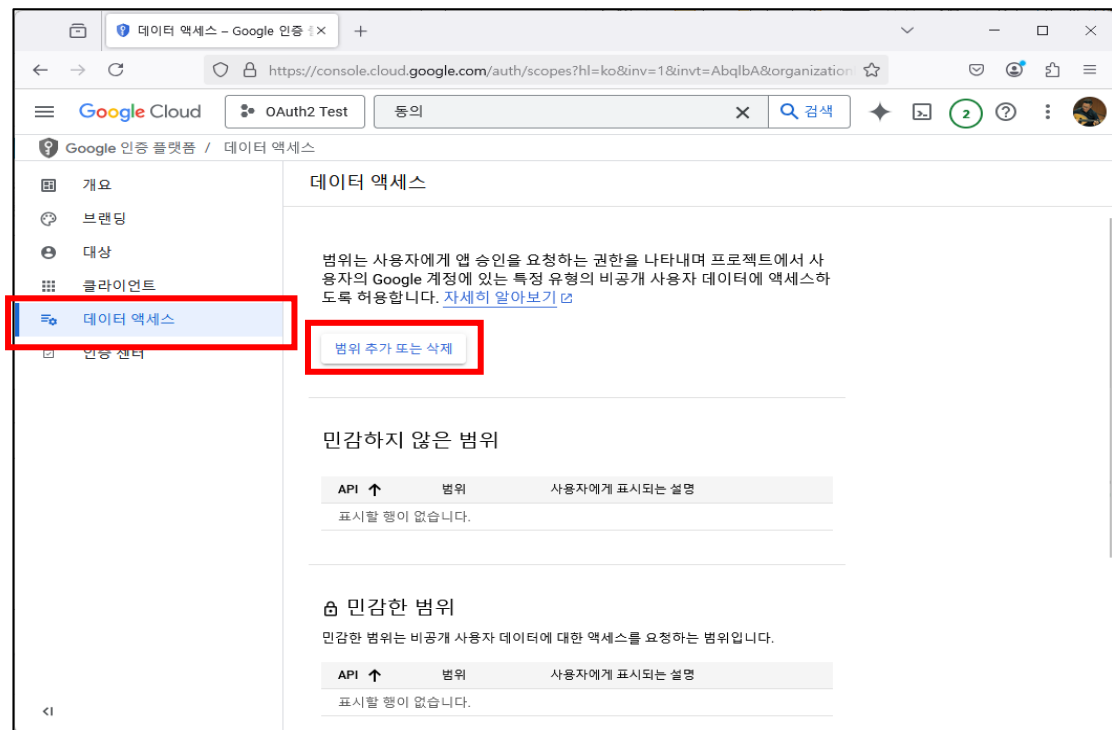
☐	이름	생성일 ↓	유형	클라이언트 ID	작업
☐	웹 클라이언트 1	2025. 2. 26.	웹 애플리케이션	131457979987-4eg2...	

A dark toast notification at the bottom center of the console states 'OAuth 클라이언트 생성됨' (OAuth client created).

Google OAuth 클라이언트 생성

6. 데이터 액세스 설정

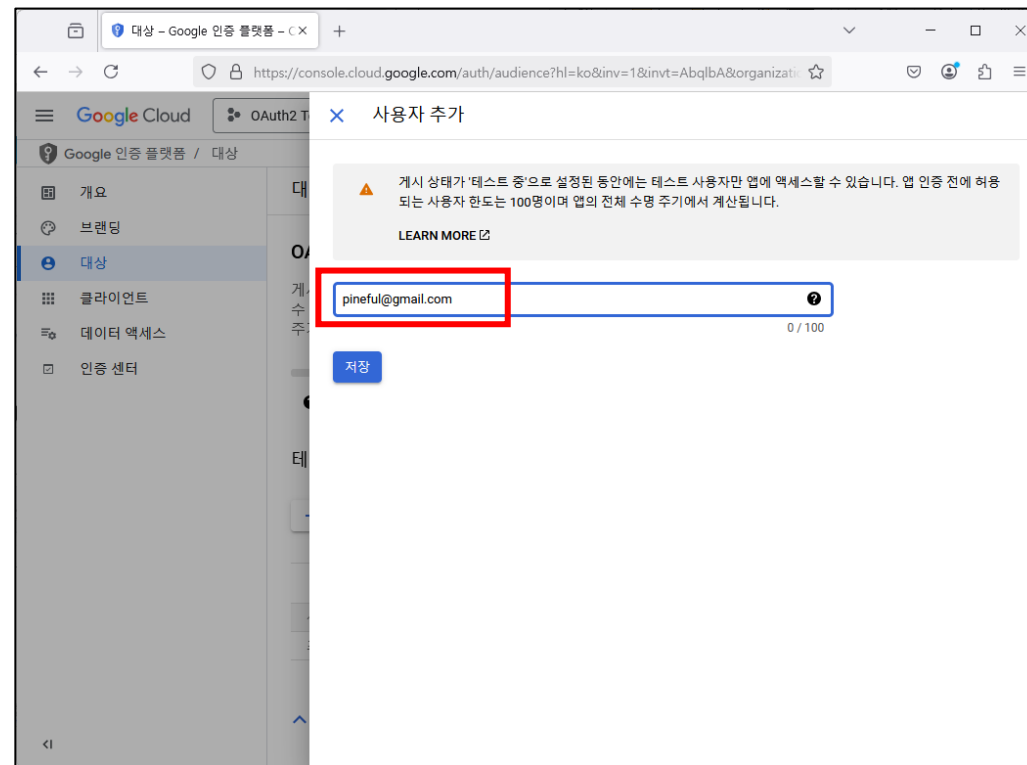
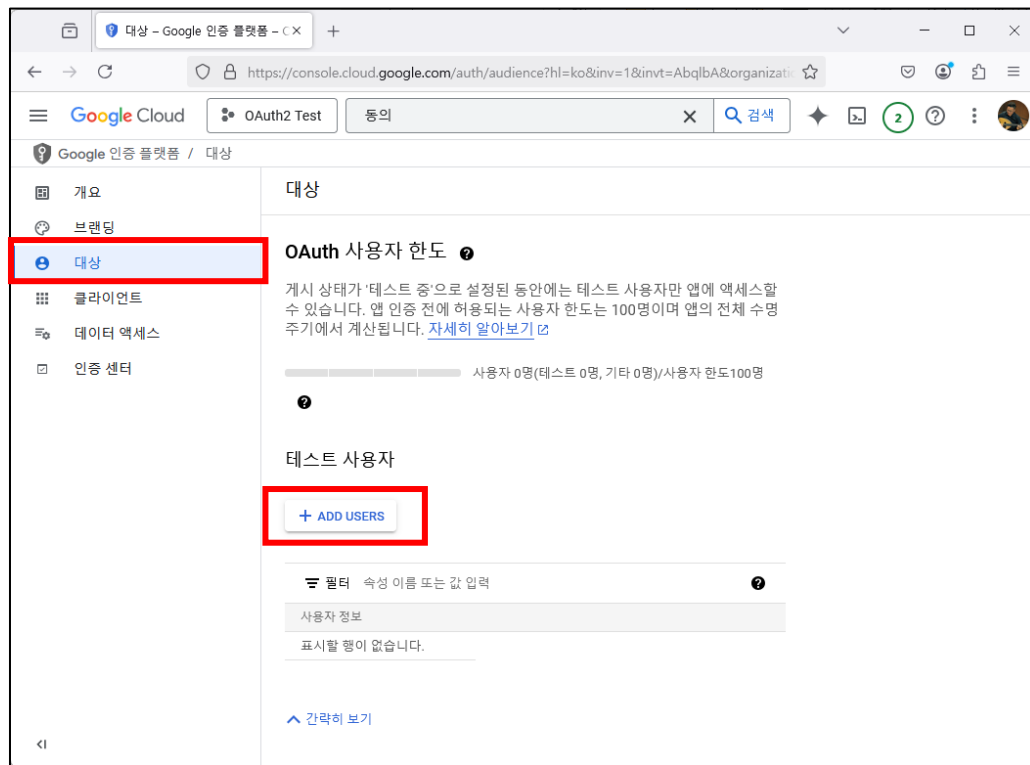
항목 추가 또는 삭제 화면으로 들어가서 2개 항목 선택 하고 [업데이트] 버튼 클릭



Google OAuth 클라이언트 생성

7. 대상 설정

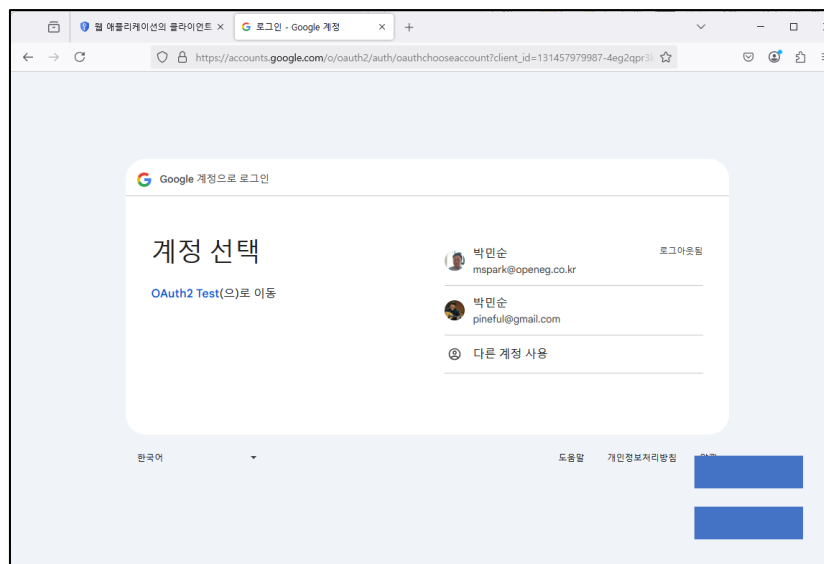
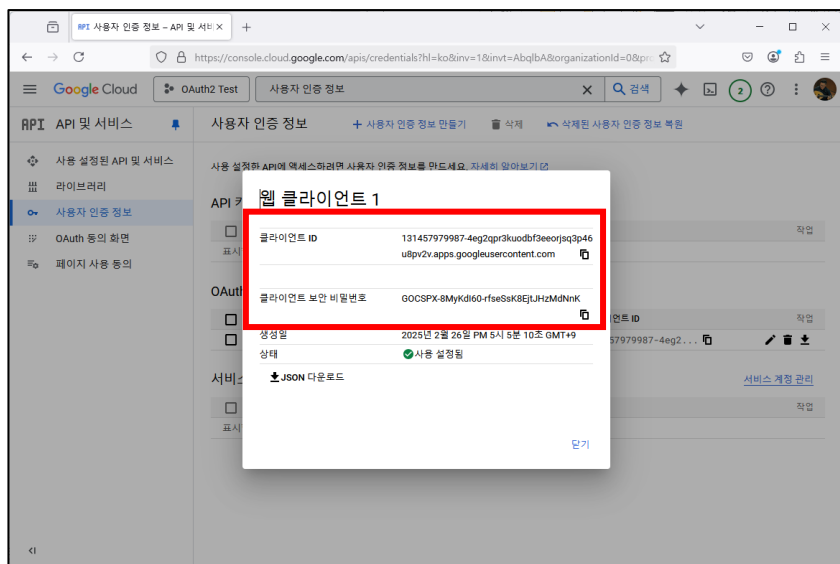
대상 > 테스트 사용자 > + ADD USERS 에 본인 메일 주소 입력



Google OAuth 클라이언트 생성

8. 설정 확인 및 테스트

사용자 인증 정보 > OAuth 2.0 클라이언트 ID > 생성한 웹 클라이언트 항목의 맨 끝 다운로드 아이콘 클릭

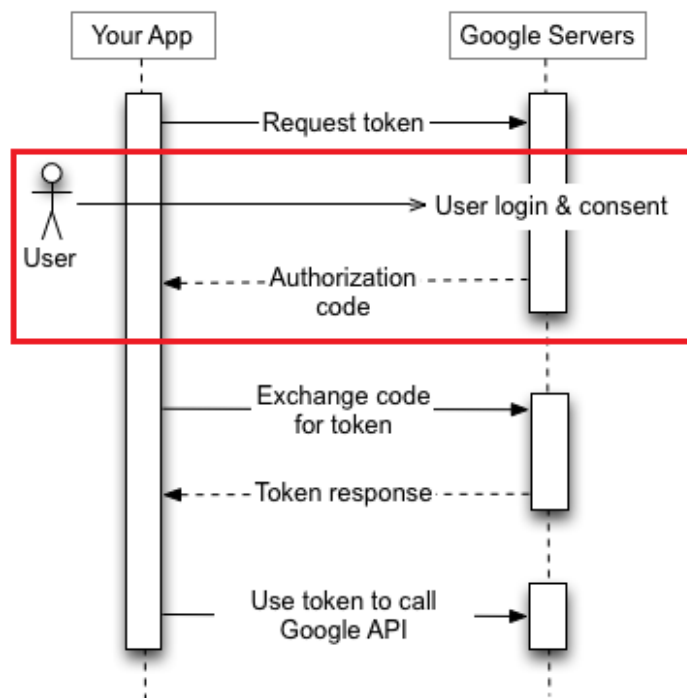


아래 URL에 클라이언트 ID를 넣어 구글 로그인 창이 뜨는지 확인

https://accounts.google.com/o/oauth2/auth?client_id=클라이언트ID&redirect_uri=http://localhost:8000/login/oauth2/code/google&response_type=code&scope=https://www.googleapis.com/auth/userinfo.email+https://www.googleapis.com/auth/userinfo.profile

Authorization Code 발급

- Google OAuth API 등록 필요
- 사용자가 구글 로그인을 마치고 나면 redirect_uri에 Authorization Code를 응답해 줌



Authorization Code 발급

- 구축되어 있는 LAB 환경에 소스 추가하여 OAuth2 연동 테스트 구현
- LAB 소스코드에서 `src/main/resources/application.properties` 파일에 클라이언트 ID, 클라이언트 Secret, 토큰, 리소스 URI 등 내용 추가

```
application.properties
1 # Server
2 server.port=8000
3 #server.error.whitelabel.enabled=false
4 #server.error.include-exception=true
5 #server.error.include-stacktrace=always
6
7 spring.web.resources.static-locations=classpath:/static/
8
9 # thymeleaf
10 spring.thymeleaf.prefix=classpath:templates/
11 spring.thymeleaf.check-template-location=true
12 spring.thymeleaf.suffix=.html
13 spring.thymeleaf.mode=HTML5
14 spring.thymeleaf.cache=false
15
16 logging.config=classpath:log4j2.xml
17
18 # OAuth2 Test
19 oauth2.google.client-id=131457979907-4eg2qpr3ku7t1t1isq3n46u8pv2v-apps.googleusercontent.com
20 oauth2.google.client-secret=GOCSPX-8MyKdI60-rfs
21 oauth2.google.redirect-uri=http://localhost:8080/login/oauth2/code/google
22 oauth2.google.token-uri=https://oauth2.googleapis.com/token
23 oauth2.google.resource-uri=https://www.googleapis.com/oauth2/v2/userinfo
24
```

발급받은 클라이언트 ID 값
발급받은 클라이언트 Secret 값
입력한 리디렉트 URI 값

Authorization Code 발급

- 컨트롤러

- 기존 로그인 컨트롤러 소스코드인 LoginController.java 끝에 코드 추가

- kr.co.openeg.lab.login.controller.LoginController.java

승인된 리디렉션 URI ⓘ

웹 서버의 요청에 사용

URI 1 *
http://localhost:8080/login/oauth2/code/google

```
55         mav.addObject("userId", userId);
56         mav.addObject("errCode", 1); // not exist userId in database
57         mav.setViewName("index");
58         return mav;
59     }else {
60         session.setAttribute("userId", userId);
61         session.setAttribute("userName", loginCheckResult.getUserName());
62         mav.setViewName("redirect:/board/main");
63
64         return mav;
65     }
66 }
67
68 @RequestMapping("/logout")
69 public String logout(HttpSession session){
70     session.invalidate();
71     return "redirect:login";
72 }
73
74 @RequestMapping("/login/oauth2/code/{registrationId}")
75 public ModelAndView googleLogin(@RequestParam(required = false) String code, @PathVariable String registrationId) {
76     service.socialLogin(code, registrationId);
77
78     ModelAndView mav = new ModelAndView();
79     mav.setViewName("redirect:/");
80     return mav;
81 }
82 }
```

kr.co.openeg.lab.login.controller.LoginController.java

Authorization Code 발급

- 서비스

- 기존 로그인 서비스 소스코드인 LoginService.java 끝에 코드 추가
- kr.co.openeg.lab.loginService.LoginService.java

```
1 package kr.co.openeg.lab.login.service;
2
3
4 import org.springframework.beans.factory.annotation.Autowired;
5
10
11 @Service
12 public class LoginService {
13
14     @Autowired
15     private LoginDao dao;
16
17     public MemberModel checkUserId(String userId) {
18         return dao.selectUserId(userId);
19     }
20
21     public MemberModel checkUserId(LoginSessionModel loginModel) {
22         return dao.selectUserId(loginModel);
23     }
24
25     public void socialLogin(String code, String registrationId) {
26         System.out.println("code = " + code);
27         System.out.println("registrationId = " + registrationId);
28     }
29
30 }
```

Authorization Code 발급

- 로그인 필터

- 로그인되어 있지 않아도 /login/oauth2 이하 URL 접근이 가능하도록 SessionInterceptor.java 소스코드 수정

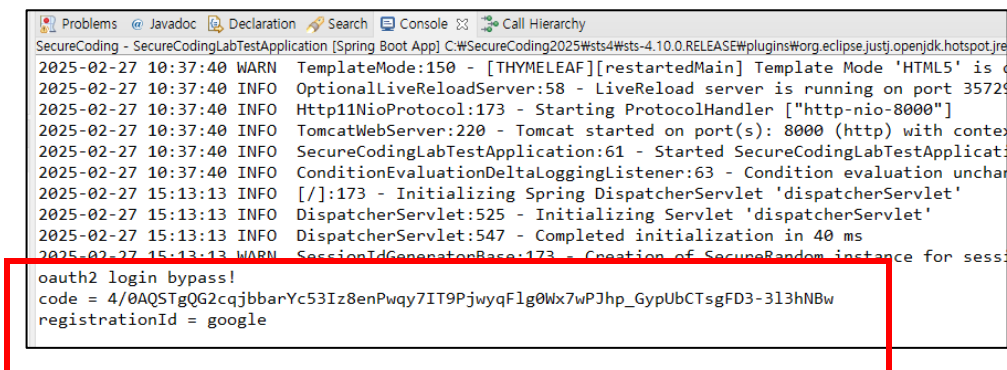
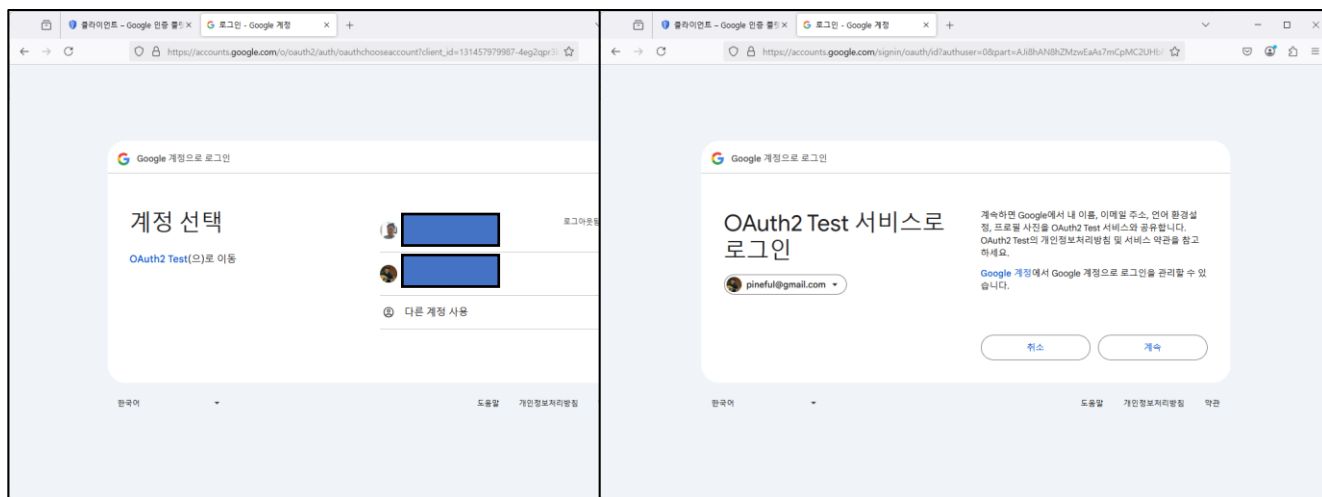
- kr.co.openeg.lab.common.interceptor.SessionInterceptor.java

```
21
22     // pass through when access login.do, join.do
23
24     if(request.getRequestURI().equals("/login")||
25        request.getRequestURI().equals("/rest/login")||
26        request.getRequestURI().equals("/vanilla/login")||
27        request.getRequestURI().equals("/member/join")){
28         if(userId != null){
29             response.sendRedirect(request.getContextPath() + "/board/list");
30             return true;
31         } else {
32             return true;
33         }
34     }
35
36     // bypass OAuth2 authentication URI
37     if(request.getRequestURI().startsWith("/login/oauth2")) {
38         System.out.println("oauth2 login bypass!");
39         return true;
40     }
41
42     //
43     // where other pages
44     if(userId == null){
45         response.sendRedirect(request.getContextPath() + "/login");
46         HttpSession session=request.getSession();
47         session.setAttribute("errCode", "4");
48         return false;
49     } else {
```

Authorization Code 발급

- 앞서 시도해봤던 URL을 입력하여 구글 로그인 창이 나타나며, 계정을 선택하고 진행하면 STS 콘솔에 메시지가 제대로 출력되면 성공

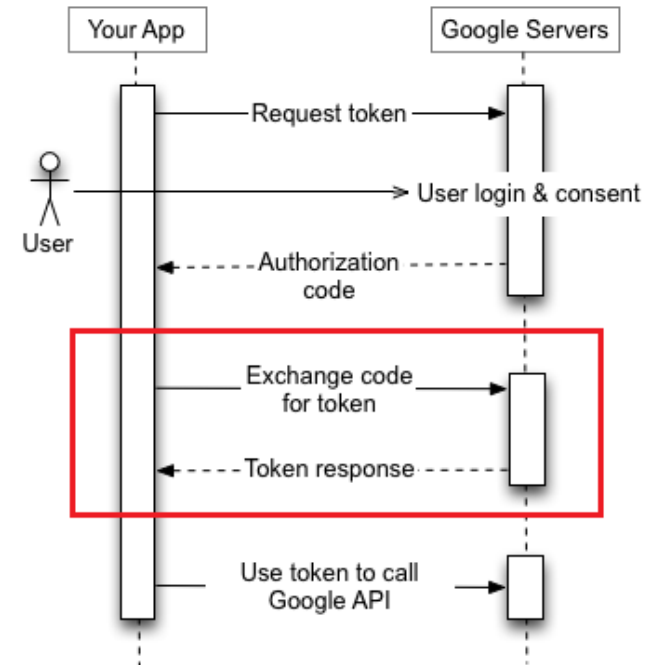
`https://accounts.google.com/o/oauth2/auth?client_id=클라이언트ID&redirect_uri=http://localhost:8000/login/oauth2/code/google&response_type=code&scope=https://www.googleapis.com/auth/userinfo.email+https://www.googleapis.com/auth/userinfo.profile`



Access Token 발급

- 발급받은 Authorization Code를 통해 Access Token 발급
- 서비스 (코드 수정)

```
33 }
34
35 public MemberModel checkUserId(LoginSessionModel loginModel) {
36     return dao.selectUserId(loginModel);
37 }
38
39 private final Environment env;
40 private final ObjectMapper objectMapper = new ObjectMapper();
41 private final RestTemplate restTemplate = new RestTemplate();
42
43 public LoginService(Environment env) {
44     this.env = env;
45 }
46 public void socialLogin(String code, String registrationId) {
47     String accessToken = getAccessToken(code, registrationId);
48     System.out.println("accessToken = " + accessToken);
49 }
50
51 private String getAccessToken(String authorizationCode, String registrationId) {
52     String clientId = env.getProperty("oauth2." + registrationId + ".client-id");
53     String clientSecret = env.getProperty("oauth2." + registrationId + ".client-secret");
54     String redirectUri = env.getProperty("oauth2." + registrationId + ".redirect-uri");
55     String tokenUri = env.getProperty("oauth2." + registrationId + ".token-uri");
56
57     MultiValueMap<String, String> params = new LinkedMultiValueMap<>();
58     params.add("code", authorizationCode);
59     params.add("client_id", clientId);
60     params.add("client_secret", clientSecret);
61     params.add("redirect_uri", redirectUri);
62     params.add("grant_type", "authorization_code");
63
64     HttpHeaders headers = new HttpHeaders();
65     headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);
66
67     HttpEntity entity = new HttpEntity(params, headers);
68
69     ResponseEntity<String> response = restTemplate.exchange(tokenUri, HttpMethod.POST, entity, String.class);
70     try {
71         JsonNode accessTokenNode = objectMapper.readTree(response.getBody());
72         return accessTokenNode.get("access_token").asText();
73     } catch (IOException e) {
74         throw new RuntimeException("Failed to parse access token response", e);
75     }
76 }
77
78 }
79
```



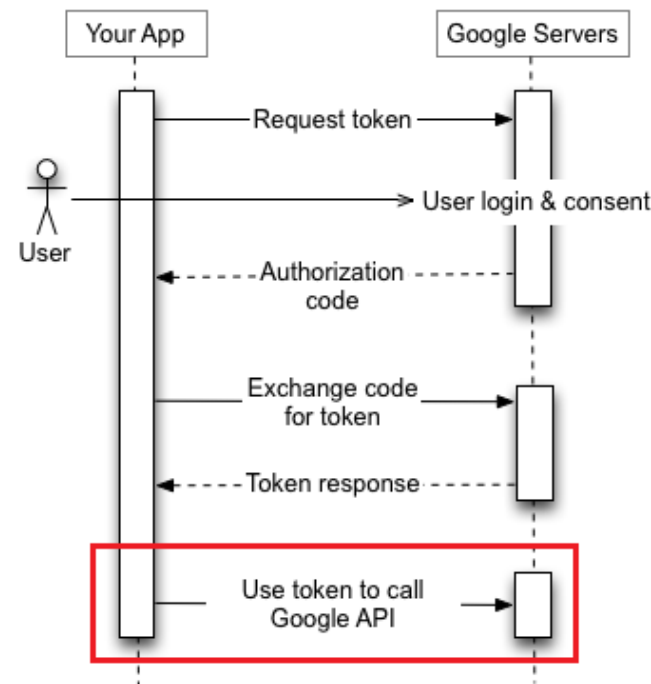
accessToken 발급 성공 콘솔 화면

```
SecureCoding - SecureCodingLabTestApplication [Spring Boot App] C:\SecureCoding\2025\wsts4\wsts-4.10.0.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.15.0.2.v20210201-0955\jre\bin\java.exe (2025. 2. 27. 오후 3:28:04)
2025-02-27 15:28:21 INFO OptionalLiveReloadServer:58 - LiveReload server is running on port 35729
2025-02-27 15:28:21 INFO Http11NioProtocol:173 - Starting ProtocolHandler ["http-nio-8000"]
2025-02-27 15:28:21 INFO TomcatWebServer:220 - Tomcat started on port(s): 8000 (http) with context path ''
2025-02-27 15:28:21 INFO SecureCodingLabTestApplication:61 - Started SecureCodingLabTestApplication in 3.253 seconds (JVM running for 15.57)
2025-02-27 15:28:51 INFO [/]:173 - Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-02-27 15:28:51 INFO DispatcherServlet:525 - Initializing Servlet 'dispatcherServlet'
2025-02-27 15:28:51 INFO DispatcherServlet:547 - Completed initialization in 1 ms
oauth2 login bypass!
accessToken = ya29.a0AeXRPp4V7qUB7KdmiomBWLtBt7c400p3t9IhL1pYJAN5n8rQZN-Ie9DXS_wLeipb7L0iuH78Um6oTb-51Wv5rD7os8j2cbedpVSNgRv2HdXaVq_BPyRh6_RFeWPEO8mhbHUEMf
2025-02-27 15:28:56 WARN FragmentProcessor:1 - You don't need to put the layout:fragment/data-layout-fragment attribute into the <head> section - the dec
```

Resource Server에서 유저정보 받기

- 발급받은 Authorization Code를 통해 Access Token 발급
- 서비스 (코드 수정)

```
46 public void socialLogin(String code, String registrationId) {
47     String accessToken = getAccessToken(code, registrationId);
48     JsonNode userResourceNode = getUserResource(accessToken, registrationId);
49     System.out.println("userResourceNode = " + userResourceNode);
50
51     String id = userResourceNode.get("id").asText();
52     String email = userResourceNode.get("email").asText();
53     String nickname = userResourceNode.get("name").asText();
54     System.out.println("id = " + id);
55     System.out.println("email = " + email);
56     System.out.println("nickname = " + nickname);
57 }
58
86 private JsonNode getUserResource(String accessToken, String registrationId) {
87     String resourceUri = env.getProperty("oauth2."+registrationId+".resource-uri");
88
89     HttpHeaders headers = new HttpHeaders();
90     headers.set("Authorization", "Bearer " + accessToken);
91     HttpEntity entity = new HttpEntity(headers);
92     return restTemplate.exchange(resourceUri, HttpMethod.GET, entity, JsonNode.class).getBody();
93 }
94 }
```



최종적으로 유저정보 수신 성공

```
SecureCoding - SecureCodingLabTestApplication [Spring Boot App] C:\SecureCoding2025\wsts4\wsts-4.10.0.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.15.0.2.v20210201-0955\jre\bin\javaw.exe (2025. 2. 27. 오후 3:41:21)
2025-02-27 15:41:26 INFO SecureCodingLabTestApplication:61 - Started SecureCodingLabTestApplication in 2.813 seconds (JVM running for 4.132)
2025-02-27 15:41:40 INFO [/]:173 - Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-02-27 15:41:40 INFO DispatcherServlet:525 - Initializing Servlet 'dispatcherServlet'
2025-02-27 15:41:40 INFO DispatcherServlet:547 - Completed initialization in 1 ms
oauth2 login bypass!
userResourceNode = {"id":"117678585699554568936","email":"[REDACTED]","verified_email":true,"name":"[REDACTED]","given_name":"[REDACTED]","family_name":"[REDACTED]","picture":"[REDACTED]"}
id = 117678585699554568936
email = [REDACTED]
nickname = [REDACTED]
2025-02-27 15:41:42 WARN FragmentProcessor:1 - You don't need to put the layout:fragment/data-layout:fragment attribute into the <head> section, the de...
```

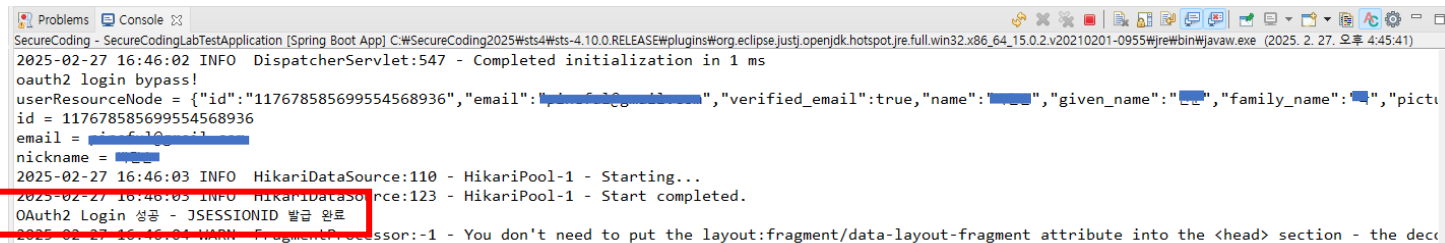
유저 정보를 토대로 계정 연동

- 원래 id 또는 email 정보를 토대로 사용자 DB와 연동 로직을 구현해야 함. 편의상 강제로 test 계정과 연동 처리.
- 컨트롤러 (코드 수정)
- 서비스 (코드 수정)

```
75 @RequestMapping("/login/oauth2/code/{registrationId}")
76 public ModelAndView googleLogin(
77     @RequestParam(required = false) String code,
78     @PathVariable String registrationId,
79     HttpServletRequest request,
80     HttpServletResponse response) {
81     service.socialLogin(code, registrationId, request, response);
82
83     ModelAndView mav = new ModelAndView();
84     mav.setViewName("redirect:/board/main");
85     return mav;
86 }
```

- test 계정으로 연동되어 세션 ID 발급됨

```
51 public void socialLogin(String code, String registrationId, HttpServletRequest request, HttpServletResponse response) {
52     String accessToken = getAccessToken(code, registrationId);
53     JsonNode userResourceNode = getUserResource(accessToken, registrationId);
54     System.out.println("userResourceNode = " + userResourceNode);
55
56     String id = userResourceNode.get("id").asText();
57     String email = userResourceNode.get("email").asText();
58     String nickname = userResourceNode.get("name").asText();
59     System.out.println("id = " + id);
60     System.out.println("email = " + email);
61     System.out.println("nickname = " + nickname);
62
63     // 강제로 test 계정과 연동.
64     MemberModel member = checkUserId("test");
65     if (member == null) {
66         throw new RuntimeException("User not found in local DB: test");
67     }
68
69     HttpSession session = request.getSession(true);
70     session.setAttribute("userId", member.getUserId());
71     session.setAttribute("userName", nickname); // 이름을 구글 계정에서 가져온다.
72
73     // 5회 클라이언트가 세션을 사용할 수 있도록 `JSESSIONID` 쿠키 설정
74     response.setHeader("Set-Cookie", "JSESSIONID=" + session.getId() + "; Path=/; HttpOnly; Secure");
75
76     System.out.println("OAuth2 Login 성공 - JSESSIONID 발급 완료");
77 }
```

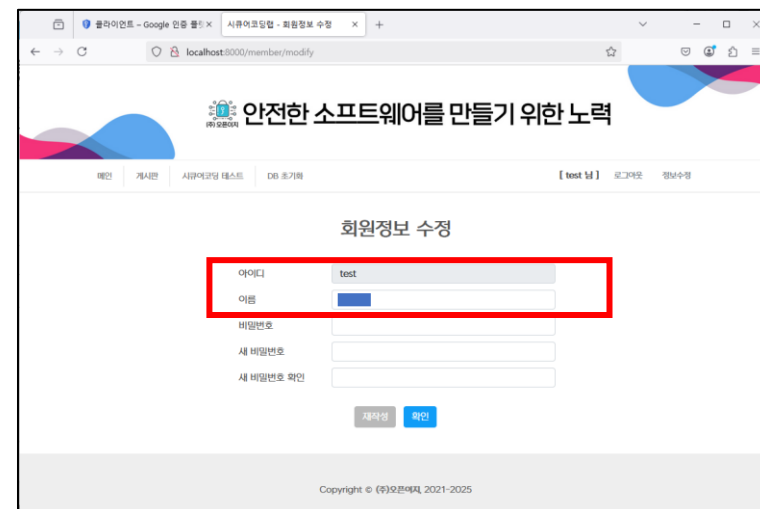
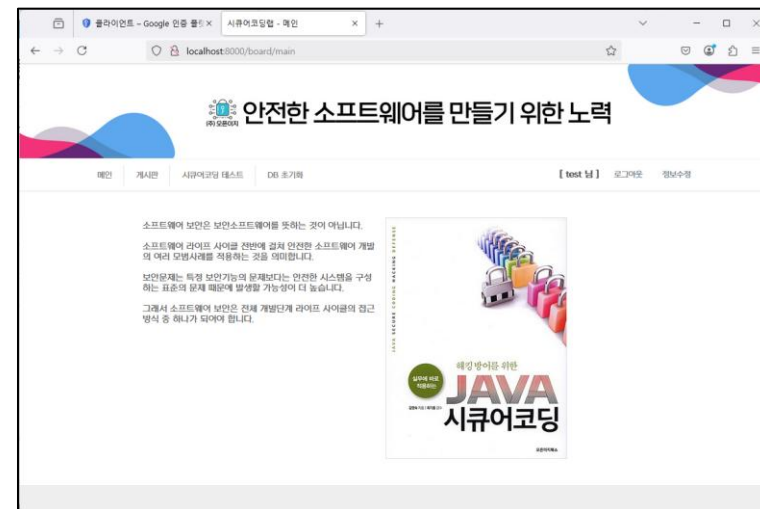


```
Problems Console
SecureCoding - SecureCodingLabTestApplication [Spring Boot App] C:\SecureCoding2025\sts4\sts-4.10.0.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.15.0.2.v20210201-0955\jre\bin\javaw.exe (2025.2.27 오후 4:45:41)
2025-02-27 16:46:02 INFO DispatcherServlet:547 - Completed initialization in 1 ms
oauth2 login bypass!
userResourceNode = {"id":"117678585699554568936","email":"[REDACTED]","verified_email":true,"name":"[REDACTED]","given_name":"[REDACTED]","family_name":"[REDACTED]","picture":"[REDACTED]"}
id = 117678585699554568936
email = [REDACTED]
nickname = [REDACTED]
2025-02-27 16:46:03 INFO HikariDataSource:110 - HikariPool-1 - Starting...
2025-02-27 16:46:03 INFO HikariDataSource:123 - HikariPool-1 - Start completed.
OAuth2 Login 성공 - JSESSIONID 발급 완료
2025-02-27 16:46:04 WARN FragmentProcessor:-1 - You don't need to put the layout:fragment/data-layout-fragment attribute into the <head> section - the dec
```


OAuth2를 통한 로그인 성공 화면

- 구글 로그인 성공하면, LAB 서비스의 test 계정으로 로그인 완료됨

- 정보수정 화면에 들어가보면, 이름 항목에 구글계정 이름이 나타남





OAuth2 인증 및 인가

01 OAuth2 이해

02 OAuth2 구현

03 OAuth2 보안 위협 및 해결 방법

OAuth2 구현시 발생할 수 있는 취약점

- 발생할 수 있는 취약점

- Redirect URI 검증 미흡
- State 파라미터 미사용
- PKCE 미적용

- 주요 취약점 사례

- Authorization Code 탈취

- 중간자 공격(MITM)이나 오픈 리다이렉트 취약점을 이용해, 인증 코드가 탈취되어 부적절한 토큰 발급 위험

- Access Token 유출

- HTTPS 미적용, 불안정한 토큰 저장(예: 로컬 스토리지, 브라우저 캐시 등)으로 토큰 노출 가능

- 토큰 재사용 및 리플레이 공격

- 만료되지 않은 토큰 또는 중복 요청 처리 미흡으로 인한 리플레이 공격, 부적절한 접근 권한 행사

- 클라이언트 시크릿 관리 취약점

- 클라이언트 시크릿(Client Secret)이 공개 클라이언트에 노출될 경우, 악의적 사용자가 토큰 발급 시도 가능

Access Token 유출 방지 전략

- 안전한 전송 및 저장
 - HTTPS/TLS 적용
 - 보안 쿠키 사용
- 토큰 관리 정책 강화
 - 짧은 유효기간
 - Refresh Token 도입
 - 토큰 취소 및 재발급
- 애플리케이션 보안 강화
 - XSS 방어: 입력값 검증, 인코딩 및 CSP(Content Security Policy) 적용
 - PKCE 사용: 공개 클라이언트의 경우, PKCE 도입으로 Authorization Code 탈취 방지



요약 정리

- ✓ OAuth2는 웹 및 애플리케이션 인증 및 권한 부여를 위한 개방형 표준 규격이다.
- ✓ Access Token 유출을 방지하기 위해 HTTPS 적용, 보안 쿠키 사용, 토큰 만료 시간 설정 등의 조치를 취해야 한다.
- ✓ OAuth2에서 발생할 수 있는 주요 취약점으로는 Redirect URI 검증 미흡, State 파라미터 미사용, PKCE 미적용 등이 있다.



참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어

The background is a vibrant blue gradient. It is decorated with various abstract elements: a large solid blue sphere in the upper left, a smaller sphere with diagonal stripes in the upper right, and another striped sphere in the lower left. Several network diagrams, consisting of interconnected nodes and lines, are scattered across the image. There are also various rectangular and trapezoidal shapes in different shades of blue. In the top left corner, there is a small icon of three horizontal lines.

감사합니다.



퀴즈

1. 다음 중 OAuth2에 대한 설명으로 맞지 않는 것은?

- 1) 인증과 인가를 분리한 프로토콜
- 3) 토큰 기반 인증 사용

- 2) 제3자 애플리케이션이 사용자의 자원에 접근하도록 허용
- 4) 비밀번호를 통해 접근 권한을 위임

2. OAuth2의 구성 요소에 대한 설명 중 잘못된 것은?

- 1) Resource Owner - 리소스 서비스 업체
- 3) Authorization Server - 사용자 인증과 권한 부여처리, 액세스 토큰 발급

- 2) Resource Server - 보호된 리소스를 보유하고 있는 서버
- 4) Client Application - 보호된 리소스에 접근하려는 애플리케이션

3. Access Token 재발급을 위한 장기 인증 수단으로, 사용자 재로그인 없이 새로운 Access Token을 발급하기 위해 사용하는 Token의 이름은 무엇인가요? (영문 소문자만)

()



퀴즈 정답

1. 다음 중 OAuth2에 대한 설명으로 맞지 않는 것은?

4) 비밀번호를 통해 접근 권한을 위임

OAuth2는 기존 OAuth 1.0의 비밀번호 유출시 취약해질 수 있는 보안을 강화하기 위해 토큰 기반 인증을 사용합니다.

2. OAuth2의 구성 요소에 대한 설명 중 잘못된 것은?

1) Resource Owner - 리소스 서비스 업체

리소스에 대한 소유 또는 접근 권한을 가지고 있는 것은 사용자입니다.

3. Access Token 재발급을 위한 장기 인증 수단으로, 사용자 재로그인 없이 새로운 Access Token을 발급하기 위해 사용하는 Token의 이름은 무엇인가요? (영문 소문자만)

(refresh)